

[Try for free](#)

Developers



API Integration

Storefront Next / Migrate from PWA Kit to St... / API Integration

API Integration Differences with PWA Kit

While both PWA Kit and Storefront Next use SCAPI for commerce functionality, they take fundamentally different approaches to API client architecture.

Architecture

Aspect	PWA Kit	Storefront Next
Package	@salesforce/commerce-sdk-react	@salesforce/storefront-next-runtime/
Underlying Client	commerce-sdk-isomorphic	openapi-fetch
Pattern	React hooks	Direct API calls
Type Generation	Manual TypeScript	Generated from specs
Bundle Size	Larger (includes React Query + SDK)	Smaller (lightweight)

Contents

Architecture

- API Call Location
- Client Initialization
- Caching
- Code Examples Comparison
- Key Similarities
- Storefront Conversion Considerations

API Call Location

Aspect	PWA Kit	Storefront Next
Reads (GET)	Inside components via hooks	In route loaders
Writes (POST/PUT)	Inside components via mutation hooks	In action routes
Authentication	Automatic via provider	Manual via middleware

Client Initialization



Where	CommerceApiProvider	createApiClient helper
Auth	Automatic token management	Middleware injects tokens

Caching

Aspect	PWA Kit	Storefront Next
Strategy	React Query cache	React Router revalidation
Invalidation	Automatic after mutations	Manual or via <code>shouldRevalidate</code>
Stale Time	Configurable (default 10s)	No built-in stale time

Code Examples Comparison

Fetching a product:

```
1 // PWA Kit - in component
2 const { data: product, isLoading } = useProduct(
3   parameters: { id: productId },
4 );
```

```
1 // Storefront Next - in loader
2 const { data } = await clients.shopperProducts.get(
3   params: {
4     path: { organizationId, id: productId },
5     query: { siteId },
6   },
7 );
```

Adding to Cart:

```
1 // PWA Kit - in component
2 const addItem = useShopperBasketsMutation("addItem");
3 addItem.mutate({
4   parameters: { basketId },
5   body: { productId, quantity },
6 });
```

[Try for free](#)

```
4   path: { organizationId, basketId },
5   query: { siteId },
6 },
7   body: [{ productId, quantity }],
8 });
```

Key Similarities

- Both use SCAPI for commerce operations.
- Both support SLAS authentication.
- Both proxy API requests through the server.
- Both provide TypeScript support.

Storefront Conversion Considerations

When migrating from PWA Kit to Storefront Next:

1. Move API calls to loaders: Replace `useProduct`, `useCategory`, and so on with loader fetches.
2. Move mutations to actions: Replace `useShopperBasketsMutation` with action routes.
3. Update parameter format: Change from `{ parameters: {} }` to `{ params: { path: {}, query: {} } }`.
4. Handle auth manually: Create auth middleware instead of relying on provider.
5. Remove React Query: No longer needed; caching handled by React Router.
6. Update error handling: Use `ApiError` class instead of hook error states.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)



Try for free



Quip

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)